VAPT TASK 2

Name: Vijayashree B
Domain: Cybersecurity (VAPT)
Date of Submission: 20.03.26

1. Vulnerability Scanning Techniques

1.1 Introduction

Vulnerability scanning is a critical component of cybersecurity assessment used to identify, evaluate, and prioritize security weaknesses in systems, networks, and applications. It helps organizations proactively detect vulnerabilities before they can be exploited by attackers. Effective scanning combines automated tools with manual validation to ensure accurate risk assessment.

1.2 Types of Vulnerability Scanning

1.2.1 Network Scanning

Network scanning is used to discover active hosts, open ports, and running services within a network. A commonly used tool is Nmap, which performs various scan types such as:

- TCP SYN Scan (stealth scanning)
- TCP Connect Scan (full connection)
- Service and version detection

This type of scanning helps identify exposed services that may be vulnerable to attacks.

1.2.2 Application Scanning

Application scanning focuses on identifying vulnerabilities in web applications and services. Tools such as Nikto and OWASP ZAP are widely used to detect:

- SQL Injection
- Cross-Site Scripting (XSS)
- Security misconfigurations
- Outdated software components

These tools simulate attacks to uncover weaknesses in application logic and configurations.

1.2.3 Authenticated vs Unauthenticated Scanning

- Unauthenticated Scanning:
  Conducted without login credentials, simulating an external attacker's perspective. It identifies publicly exposed vulnerabilities.
- Authenticated Scanning:
  Performed with valid credentials, allowing deeper inspection of system configurations, installed patches, and internal vulnerabilities.

Authenticated scans provide more accurate and comprehensive results compared to unauthenticated scans.

1.3 Vulnerability Scoring Using CVSS

The severity of vulnerabilities is assessed using the CVSS v4.0, which assigns a numerical score ranging from 0.0 to 10.0.

Score Range	Severity Level
0.0 – 3.9	Low
4.0 – 6.9	Medium

Score Range	Severity Level
7.0 – 8.9	High
9.0 – 10.0	Critical

For example, the vulnerability CVE-2017-5638 is classified as Critical, as it allows Remote Code Execution (RCE), enabling attackers to take full control of affected systems.

1.4 False Positives and Validation

Automated vulnerability scanners may generate false positives, which are incorrect or misleading results indicating vulnerabilities that do not actually exist.

Common Causes:

- Incorrect service detection
- Outdated vulnerability databases
- Misinterpretation of responses

Validation Techniques:

- Manual verification using command-line tools (e.g., curl, telnet)
- Cross-checking with multiple scanning tools
- Reviewing system configurations and logs

Proper validation ensures that only genuine vulnerabilities are reported, improving the accuracy of the assessment.

1.5 Methodology and Best Practices

A systematic approach to vulnerability scanning includes:

1. Asset Identification: Identify systems and services within the target environment
2. Scan Configuration: Select appropriate tools and scan types
3. Execution: Perform network and application scans
4. Analysis: Interpret scan results and identify vulnerabilities
5. Risk Prioritization: Use CVSS scores to rank vulnerabilities
6. Validation: Eliminate false positives through manual testing
7. Reporting: Document findings with remediation recommendations

---

1.6 Standards and Guidelines

Vulnerability scanning practices are guided by established frameworks such as:

- OWASP Testing Guide
  Provides methodologies for testing web application security.
- NIST SP 800-115
  Offers technical guidance on security testing and assessment methodologies.

1.7 Case Study: WannaCry Ransomware Attack

The WannaCry ransomware attack exploited a vulnerability in Microsoft Windows systems known as MS17-010.

This vulnerability allowed attackers to execute remote code and spread rapidly across networks. The incident highlights:

- The importance of timely patch management
- The role of vulnerability scanning in identifying critical flaws
- The need for accurate risk assessment using CVSS

1.8 Conclusion

Vulnerability scanning is an essential practice for maintaining system security. By combining automated tools, standardized scoring systems, and manual validation, organizations can effectively identify and mitigate risks. Adhering to established guidelines and continuously monitoring systems ensures a proactive defense against evolving cyber threats.

---

2. Penetration Testing Techniques

2.1 Introduction

Penetration testing, commonly known as ethical hacking, is a controlled and authorized process of simulating cyberattacks to identify and exploit security vulnerabilities in systems, networks, or applications. Unlike vulnerability scanning, penetration testing goes a step further by actively exploiting weaknesses to assess their real-world impact.

2.2 Phases of Penetration Testing

Penetration testing follows a structured approach consisting of several key phases:

2.2.1 Reconnaissance (Information Gathering)

This phase involves collecting information about the target system without direct interaction (passive) or with limited interaction (active).

Tools such as Shodan are used to gather:

- Publicly available data (OSINT)
- IP addresses and domains
- Exposed services and technologies

This phase helps in understanding the target environment before launching attacks.

2.2.2 Scanning

In this phase, the attacker identifies live systems, open ports, and vulnerabilities.

Tools such as Nessus are commonly used to:

- Detect known vulnerabilities
- Identify misconfigurations
- Analyze service versions

Scanning provides a technical map of the target system.

2.2.3 Exploitation

Exploitation involves taking advantage of identified vulnerabilities to gain unauthorized access.

A widely used framework is Metasploit, which allows testers to:

- Execute exploits
- Gain system access
- Validate vulnerabilities

This phase demonstrates the actual risk associated with vulnerabilities.

2.2.4 Post-Exploitation

Once access is gained, testers assess the impact and extent of the compromise.
Activities include:

- Privilege escalation (gaining admin/root access)
- Data extraction
- Maintaining access (persistence)

This phase evaluates how far an attacker can go after initial compromise.

2.2.5 Reporting

The final phase involves documenting all findings in a structured report.
A good report includes:

- Identified vulnerabilities
- Exploitation results
- Risk severity
- Remediation recommendations

This phase is critical for helping organizations improve their security posture.

2.3 Penetration Testing Methodologies

2.3.1 PTES (Penetration Testing Execution Standard)

The PTES provides a comprehensive framework for conducting penetration tests.
It includes:

- Pre-engagement interactions (scoping, rules of engagement)
- Intelligence gathering
- Threat modeling
- Exploitation
- Reporting

PTES ensures that penetration testing is systematic and consistent.

2.3.2 OWASP Web Security Testing Guide

The OWASP WSTG focuses specifically on web application security testing.
It provides:

- Detailed testing techniques
- Checklists for vulnerabilities
- Best practices for secure web applications

This methodology is widely used for web penetration testing.

2.4 Ethical Considerations in Penetration Testing

Penetration testing must always be conducted ethically and legally.
Key Ethical Principles:

- **Authorization:**
  Testing must only be performed with explicit permission from the system owner.
- **Defined Scope:**
  Clearly specify systems, networks, and applications to be tested.

- **Confidentiality:**
  Sensitive data accessed during testing must be protected.
- **No Damage Policy:**
  Avoid disrupting services or causing data loss.

Failure to follow ethical guidelines can result in legal consequences and reputational damage.

2.5 Key Objectives of Penetration Testing

The main objectives include:

- Identifying exploitable vulnerabilities
- Assessing real-world attack impact
- Testing security controls and defenses
- Providing actionable remediation strategies

Penetration testing helps organizations move from theoretical risk to practical risk understanding.

2.6 Learning Resources and Approach

2.6.1 Study Frameworks and Standards

- PTES
  → Understand full penetration testing lifecycle
- OWASP WSTG
  → Learn web application testing techniques

2.6.2 Practical Learning

- Practice using tools like:
  - Metasploit
  - Nessus
  - Burp Suite (for web testing)
- Set up labs using:
  - Vulnerable machines (e.g., Metasploitable)
  - Capture-the-Flag (CTF) platforms

2.6.3 Case Study Analysis

Study real-world penetration testing scenarios from organizations such as SANS Institute to understand:

- Attack techniques
- Exploitation paths
- Defensive strategies

2.7 Conclusion

Penetration testing is a vital cybersecurity practice that simulates real-world attacks to uncover exploitable vulnerabilities. By following structured methodologies such as PTES and OWASP WSTG, and adhering to strict ethical standards, organizations can effectively strengthen their security posture and defend against cyber threats.

---

3. Exploit Development Basics

3.1 Introduction

Exploit development is the process of creating code that takes advantage of vulnerabilities in software systems to demonstrate their impact. It is a crucial skill in penetration testing, as it helps validate whether identified vulnerabilities can be successfully exploited. Understanding exploit development also aids in building stronger defensive mechanisms.

3.2 Core Concepts of Exploit Development

3.2.1 Types of Exploits

Exploits target different categories of vulnerabilities depending on the nature of the flaw.

- **Buffer Overflow Attacks:**
  Occur when a program writes more data to a buffer than it can handle, overwriting adjacent memory. This can lead to arbitrary code execution.
- **SQL Injection (SQLi):**
  Exploits improper input validation in database queries, allowing attackers to manipulate or retrieve unauthorized data.
- **Cross-Site Scripting (XSS):**
  Occurs when untrusted input is not properly sanitized, allowing attackers to inject malicious scripts into web pages viewed by other users.

Example: XSS can occur when user input is displayed without proper escaping, enabling execution of malicious JavaScript in a victim's browser.

3.2.2 Exploit Writing

Exploit writing involves developing code to leverage vulnerabilities for controlled testing.
Security researchers often refer to publicly available proof-of-concept (PoC) exploits from Exploit Database, which contains:

1. Verified exploit scripts
2. Vulnerability descriptions
3. Technical details for exploitation

Basic exploits can be written using languages such as Python or C. For example:

- Crafting payloads for buffer overflow attacks
- Automating exploitation steps
- Modifying existing PoCs to fit specific environments

3.2.3 Mitigation Techniques

Modern systems implement several defenses to prevent exploitation:

- **Address Space Layout Randomization (ASLR):**
  Randomizes memory locations, making it difficult to predict where to inject malicious code.
- **Web Application Firewalls (WAFs):**
  Filter and monitor HTTP traffic to block malicious requests.
- **Patching and Updates:**
  Fix known vulnerabilities and prevent exploitation.

Understanding these mitigations is essential for both attackers (to bypass them) and defenders (to strengthen security).

3.3 Key Objectives of Exploit Development

The main objectives include:

- Understanding how vulnerabilities are exploited
- Developing controlled and safe exploit code
- Testing exploit reliability in a lab environment
- Evaluating the effectiveness of security mitigations

Exploit development bridges the gap between vulnerability identification and real-world attack simulation.

3.4 Safe Exploit Development Practices

Exploit development must always be performed in a controlled environment.
Best Practices:

- Use isolated lab environments (e.g., virtual machines)
- Avoid testing on production systems
- Follow legal and ethical guidelines
- Document all testing procedures

This ensures that exploitation activities do not cause unintended harm.

3.5 Learning Resources and Approach

3.5.1 Study Public Exploits

- Use Exploit Database to:
  - Analyze real-world exploit code
  - Understand vulnerability exploitation techniques
  - Practice modifying PoCs

3.5.2 Guided Learning Platforms

- TCM Security
  → Offers practical exploit development guides
- TryHackMe
  → Provides hands-on labs such as buffer overflow challenges

These platforms help learners gain practical, real-world skills.

3.5.3 Hands-On Practice

- Practice buffer overflow exploitation in controlled labs
- Write simple Python scripts to automate attacks
- Test exploits against intentionally vulnerable systems

Practical experience is essential to mastering exploit development.

3.6 Example Scenario: Buffer Overflow Exploit

In a buffer overflow scenario:

1. Identify vulnerable input field
2. Determine buffer size
3. Craft payload to overwrite memory
4. Inject shellcode or redirect execution

This demonstrates how improper memory handling can lead to full system compromise.

3.7 Conclusion

Exploit development is a fundamental aspect of advanced cybersecurity practices. By understanding different exploit types, learning how to write and test exploits, and studying mitigation techniques, security professionals can better assess risks and strengthen system defenses. Safe and ethical practices are essential when performing exploit development activities.